

SaveTune - Guía de Inicio Rápido

Esta guía te ayudará a poner en marcha tanto el backend como la app Android en menos de 30 minutos.

Pasos Resumidos

1. Backend (15 min) → 2. Android (15 min) → 3. ¡Listo! 🎉

1 Backend (Node.js + Vercel)

Setup Local (5 min)

```
bash

# Crear carpeta del proyecto
mkdir savetune-backend
cd savetune-backend

# Copiar archivos del repositorio
# - package.json
# - vercel.json
# - .env.example
# - Carpeta src/ completa

# Instalar dependencias
npm install

# Crear archivo .env
cp .env.example .env

# Ejecutar en modo desarrollo
npm run dev

# ✅ Backend corriendo en http://localhost:3000
```

Probar Backend (2 min)

```
bash

# Test 1: Health check
curl http://localhost:3000/health

# Test 2: Playlist (usa una playlist pública de Spotify)
curl "http://localhost:3000/api/playlist?url=https://open.spotify.com/playlist/37i9dQZF1DXcBWIGoYBM5M"

# ✅ Si ves JSON con canciones, funciona!
```

Deploy a Vercel (8 min)

```
bash

# 1. Instalar CLI de Vercel
npm install -g vercel

# 2. Login (abre navegador)
vercel login

# 3. Deploy
vercel

# Preguntas interactivas:
# - Set up and deploy? [Y]
# - Which scope? [Tu cuenta]
# - Link to existing project? [N]
# - Project name? [savetune-backend]
# - Directory? [./]
# - Override settings? [N]

# 4. Deploy a producción
vercel --prod

# ✅ Tu API está en: https://savetune-backend-xxx.vercel.app
# ⚠️ GUARDA ESTA URL - la necesitarás para Android
```

🎉 ¡Backend listo! Tu API ya está funcionando en la nube.

2 Android App (Kotlin + Compose)

Crear Proyecto (3 min)

Opción A: Desde Android Studio

1. Abrir Android Studio
2. New Project → Empty Activity (Compose)
3. Name: SaveTune
4. Package: com.tuapp.savetune
5. Language: Kotlin
6. Minimum SDK: API 24 (Android 7.0)
7. Build configuration: Kotlin DSL (build.gradle.kts)

Opción B: Desde terminal

```
bash
```

```
# Si tienes Android Studio instalado:
```

```
studio.sh
```

```
# Luego sigue pasos de Opción A
```

Configurar Dependencias (3 min)

1. Reemplazar `app/build.gradle.kts` con el archivo proporcionado
2. Sync Gradle (botón "Sync Now" en Android Studio)
3. Esperar a que descargue dependencias (~2-3 min primera vez)

Copiar Código (5 min)

Copiar estos archivos a tu proyecto Android:

```
app/src/main/java/com/tuapp/savetune/
```

```
|— data/
|   |— api/
|   |   |— ApiService.kt
|   |   |— RetrofitClient.kt
|   |— models/
|   |   |— Song.kt
|   |   |— PlaylistResponse.kt
|   |   |— DownloadResponse.kt
|   |— repository/
|   |   |— SongRepository.kt
|— ui/
|   |— screens/
|   |   |— HomeScreen.kt
|   |   |— PlaylistScreen.kt
|   |— components/
|   |   |— SongItem.kt
|   |— theme/
|   |   |— Theme.kt (ya existe)
|— viewmodel/
|   |— PlaylistViewModel.kt
|— MainActivity.kt
```

Configurar Backend URL (1 min)

En `RetrofitClient.kt`, línea 12:

kotlin

// Cambiar esto:

```
private const val BASE_URL = "https://tu-backend.vercel.app/api/"
```

// Por tu URL de Vercel:

```
private const val BASE_URL = "https://savetune-backend-xxx.vercel.app/api/"
```

Configurar Permisos (1 min)

Asegúrate que `AndroidManifest.xml` tenga:

xml

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission
    android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="32" />
```

Build & Run (2 min)

bash

Desde Android Studio:

1. Click en el triángulo verde 

2. Selecciona un emulador o dispositivo

3. Espera a que compile y se instale

Desde terminal:

```
./gradlew installDebug
```

```
adb shell am start -n com.tuapp.savetune/.MainActivity
```



¡App lista! Ya puedes pegar URLs de Spotify y descargar canciones.

3 Testing End-to-End

Test Completo (3 min)

1. Abre la app en Android
 - Verás pantalla "SAVETUNE"
2. Pega una URL de playlist

```
https://open.spotify.com/playlist/37i9dQZF1DXcBWIGoYBM5M
```

3. Click en "ANALYZE PLAYLIST"

- La app envía request a tu backend
- Backend scrape Spotify con Puppeteer
- Responde con lista de canciones
- App muestra las canciones

4. Click en el botón de descarga de una canción

- App busca canción en YouTube
- Backend descarga el audio
- App guarda el MP3 localmente
- Verás el progreso (0% → 100%)

✅ Si todo esto funciona, ¡proyecto completo!

Troubleshooting Rápido

Backend no responde

```
bash

# Ver logs de Vercel
vercel logs

# Verificar que está desplegado
curl https://tu-backend.vercel.app/health
```

Android no conecta al backend

```
kotlin

// Si pruebas con localhost en emulador:
private const val BASE_URL = "http://10.0.2.2:3000/api/"

// En dispositivo real con backend local:
// Usa la IP de tu PC, ejemplo:
private const val BASE_URL = "http://192.168.1.100:3000/api/"
```

Error de compilación en Android

```
bash
```

```
# Limpiar proyecto
```

```
./gradlew clean
```

```
./gradlew build
```

```
# O desde Android Studio:
```

```
Build > Clean Project
```

```
Build > Rebuild Project
```

Puppeteer timeout en Vercel

- Playlists muy grandes pueden exceder 10s
 - Usa playlists más pequeñas para testing
 - En producción, considera paginar resultados
-

Checklist Final

Backend

- ☐ `npm install` ejecutado sin errores
- ☐ Backend corre local en :3000
- ☐ Health check responde
- ☐ Endpoint `/api/playlist` devuelve canciones
- ☐ Desplegado en Vercel
- ☐ URL de producción guardada

Android

- ☐ Proyecto creado en Android Studio
- ☐ Dependencias sincronizadas
- ☐ Código copiado a carpetas correctas
- ☐ URL del backend configurada
- ☐ Permisos agregados al Manifest
- ☐ App compila sin errores
- ☐ App corre en emulador/dispositivo

Testing

- ☐ Puedo pegar URL de Spotify
- ☐ Lista de canciones se carga
- ☐ Puedo descargar una canción
- ☐ Archivo MP3 se guarda localmente
- ☐ Progress bar funciona

Siguiente Nivel

Una vez que todo funcione:

1. Mejora el UI

- Personaliza colores en `Theme.kt`
- Agrega animaciones
- Mejora iconografía

2. Agrega Features

- Cache con Room Database
- Player de audio integrado
- Gestión de múltiples playlists
- Búsqueda de canciones

3. Optimiza

- Reduce tamaño de APK con ProGuard
- Implementa paginación
- Agrega retry logic
- Mejora manejo de errores

4. Publica

- Crea cuenta de Google Play Developer
- Diseña assets (ícono, capturas)
- Sube APK firmado
- ¡Lanza tu app!

Recursos Útiles

Documentación

- [Vercel Docs](#)
- [Puppeteer Docs](#)
- [Compose Docs](#)
- [Retrofit Guide](#)

Tutoriales

- [Kotlin Coroutines](#)
- [Jetpack Compose Tutorial](#)
- [Node.js Express](#)

Comunidad

- [r/androiddev](#)
 - [r/node](#)
 - [Stack Overflow](#)
-

Tips Pro

1. **Usa DevTools:** F12 en Spotify web para ver requests
 2. **Logea todo:** `console.log` en backend, `Log.d` en Android
 3. **Emulador vs Real:** Siempre prueba en dispositivo real antes de publicar
 4. **Versionado:** Usa Git desde el día 1
 5. **Backups:** Vercel y GitHub tienen versión gratis
-

¡Felicitaciones!

Si llegaste hasta aquí, tienes una app Android completa que:

-  Scrape Spotify con Puppeteer
-  Descarga audio de YouTube
-  UI moderna con Jetpack Compose
-  Arquitectura MVVM profesional
-  Backend escalable en Vercel

¡Ahora a mejorarla y publicarla! 